



BITS 3423

INFORMATION TECHNOLOGY SECURITY

SEMESTER: 2-2025/2026

PROJECT REPORT

PROJECT TITLE:

**ALMS SECURE WEB – EMPLOYEES ATTENDANCE AND LEAVE
MANAGEMENT SYSTEM**

LECTURER NAME:

TS. DR. ZURINA BINTI SA'AYA

GROUP NUMBER: 12

NAME	MATRIC NO
WARDINA NAJIHAH BINTI RUSMADI	B032310309
NURUL FATNIN HAIDAH BINTI SHAMARUL ASRIN	B032310463
NURUL FATIHAH BINTI ZUKIFLI	B032310573

29 April 2026

TABLE OF CONTENTS

1. Introduction.....	5
1.1. Project Background	5
1.2. Problem Statement	5
1.3. Objectives	6
1.4. Scope	6
1.5. Project Significance	7
1.6. Task Distribution	7
2. System Architecture	8
2.1. VM Infrastructure and Network	8
2.2. Technology Stack	9
2.3. Data Flow	10
3. Implementation	10
3.1. VM Setup and Network Configuration	10
3.2. Apache, SSL/TLS and Firewall (VM1)	14
3.3. MySQL Setup (VM2)	18
4. Database Schema	20
4.1. Key Relationships	20
5. Application Modules.....	22
5.1. Database Connection (db.php)	22
5.2. Register Module (register.php)	23
5.3. Login Module (login.php)	25
5.4. Dashboard Module (dashboard.php)	27
5.5. Attendance Module (attendance.php)	30
5.6. Leave Management Modules	32
5.7. Sidebar and Logout (sidebar.php/ logout.php)	36
6. OWASP Security Implementation	37
6.1. A01 – Broken Access Control	37
6.2. A02 – Cryptographic Failures	38
6.3. A03 – SQL Injection Prevention	41
6.4. A03 – XSS Prevention	43
6.5. A07 – Identification and Authentication Failures	45
7. Troubleshooting.....	46
8. Conclusion	48

LIST OF FIGURES

Figure 3.1.1: VirtualBox Manager showing both VM1-WebServer and VM2-DBServer running (green status).....	11
Figure 3.1.2: VM1 Adapter 1 configured as NAT.....	11
Figure 3.1.3: VM1 Adapter 2 configured as Host-Only on vboxnet0 (192.168.56.0/24).....	12
Figure 3.1.4: VM2 Network Adapter Settings (Host-Only Only).....	13
Figure 3.1.5: Ping Test — VM1 to VM2 Connectivity.....	14
Figure 3.2.1: Proceed to Site.....	16
Figure 3.2.2: Certificate Details.....	17
Figure 3.2.3: VM2 UFW Rules.....	18
Figure 3.3.1: Remote Connection from VM1.....	19
Figure 4.1.1: Show Databases.....	21
Figure 4.1.2 : bcrypt Hash.....	22
Figure 5.2.1 Register Page.....	25
Figure 5.3.1: Login Page.....	27
Figure 5.4.1:Staff Dashboard.....	29
Figure 5.4.2: Manager Dashboard.....	29
Figure 5.4.3: Admin Dashboard.....	30
Figure 5.6.1 Attendance Page.....	32
Figure 5.6.1: Apply Leave Page.....	35
Figure 5.6.2: Manager Leave Approval.....	35
Figure 5.6.3: Admin Final Approval.....	36
Figure 5.6.4 Leave Balance Page.....	36
Figure 6.1.1: Unauthenticated Redirect.....	38
Figure 6.2.1: bcrypt Hash Stored in Database.....	40
Figure 6.2.2: HTTPS Certificate Details.....	41
Figure 6.3.1: SQL Injection Attempt Fails.....	43
Figure 6.4.1 XSS: Script Tag Rendered as Plain Text.....	44

Figure 6.5.1: Session Destroyed on Logout (Cookie Flags)	46
--	----

1. Introduction

1.1. Project Background

The use of secure web applications has become increasingly important in modern organisations, especially for systems involving employee management and sensitive information. Traditional attendance and leave management methods often rely on manual processes, spreadsheets, or unsecured systems, which may lead to data inaccuracies, unauthorised access, and inefficient workforce management.

To address these issues, the ALMS Secure Web the Employee Attendance and Leave Management System was developed as a secure multi-tier web application that integrates attendance tracking, leave management, and secure authentication within a virtualised environment.

This project focuses not only on application development but also on secure infrastructure implementation. The system was deployed using two isolated virtual machines consisting of a Web Server (VM1) and a Database Server (VM2). Security measures such as HTTPS encryption, SSH remote administration, firewall configuration, and OWASP Top 10 protections were implemented to ensure secure communication and data handling throughout the system.

1.2. Problem Statement

Organisations that rely on manual or unsecured systems for managing employee attendance and leave face a number of recurring challenges:

- i. Inaccurate attendance records due to manual data entry errors, leading to incorrect salary calculations and disputes.
- ii. Lack of access control in existing systems allows unauthorised users to view or modify sensitive employee data.
- iii. Passwords and sensitive records stored in plaintext are vulnerable to data breaches if the system is compromised.
- iv. Absence of encrypted communication exposes data in transit to interception.
- v. High administrative workload from manual leave processing reduces HR

efficiency and increases the risk of errors.

ALMS Secure Web directly addresses each of these problems through secure system design, role-based access control, encrypted communication, and automated leave management.

1.3. Objectives

- i. To develop a secure web-based Employee Attendance and Leave Management System.
- ii. To implement secure communication and infrastructure using SSL/TLS, SSH, and an isolated database architecture.
- iii. To apply OWASP Top 10 security practices including SQL injection prevention, session security, password hashing, and XSS protection.
- iv. To demonstrate secure integration between the Web Server and Database Server within a virtualised environment.

1.4. Scope

The scope of ALMS Secure Web covers the following modules and infrastructure components:

- i. Register Module
new user account creation with role selection, email validation, and bcrypt password hashing.
- ii. Login Module
secure credential verification with session management and login attempt throttling.
- iii. Dashboard Module
role-based display of attendance records and leave balances accessible only after login.
- iv. Attendance Module
employee clock-in and clock-out with automatic status tagging (Present, Late, Half Day, Overtime).
- v. Leave Management Module
leave application, approval/rejection workflow, balance tracking, and leave history.

- vi. OWASP Security Controls
A01 (Access Control), A02 (Cryptographic Failures), A03 (SQL Injection + XSS), A07 (Session Management).
- vii. Virtual Machine Infrastructure
two isolated Ubuntu Server 22.04 VMs with Host-Only + NAT networking, UFW firewall, and SSH/SFTP hardening.

1.5. Project Significance

The ALMS Secure Web system provides the following benefits to organisations:

- i. Secure HR Management
role-based access control ensures that employees, managers, and administrators can only access data relevant to their role, preventing unauthorised data exposure.
- ii. Data Protection
all passwords are hashed using bcrypt before storage and all data in transit is protected by TLS encryption, significantly reducing the risk of data breaches.
- iii. Efficient Attendance Tracking
automated clock-in/clock-out with real-time status updates eliminates manual data entry errors and reduces the administrative burden on HR staff.
- iv. Secure Authentication
session fixation prevention, secure cookie flags, and login attempt throttling protect user accounts from common authentication attack.

1.6. Task Distribution

Member	Primary Responsibilities	Deliverables
NURUL FATNIN HAIDAH (B032310463)	Project Leader & Backend Developer. Designed and developed PHP web application modules including Register, Login, Dashboard, Attendance, and Leave Management. Implemented database integration, user role management, and	PHP source code, database integration scripts, attendance & leave modules, authentication system, backend documentation.

	application logic. Applied OWASP A01 (Access Control) and A03 (Input Validation/SQL Injection Prevention) controls within the application.	
WARDINA NAJIHAH (B032310309)	Infrastructure & Server Administrator. Configured VirtualBox environment, deployed Ubuntu Server VMs, implemented Apache Web Server, SSL/TLS certificates, SSH/SFTP access, UFW firewall configuration, network setup (NAT & Host-Only), MySQL server deployment, and system hardening. Conducted connectivity and deployment testing.	Virtual machine setup, Apache SSL configuration, firewall documentation, SSH configuration, network topology diagram, deployment and testing evidence.
NURUL FATIHAH (B032310573)	Security & Documentation Specialist. Implemented OWASP A02 (Password Hashing), A07 (Session Management), and A03 (XSS Protection). Developed admin functions, reporting features, CSV export module, performed security testing, prepared project documentation, presentation materials, and final demonstration.	Security implementation code, session management module, admin panel, reporting/export module, final report, presentation slides, demo script.

2. System Architecture

2.1. VM Infrastructure and Network

The system uses two VMs connected over a Host-Only private network (192.168.56.0/24). VM1 additionally has a NAT adapter to allow the host browser to reach the web application over HTTPS. VM2 has no NAT adapter, ensuring it is completely isolated from any public network.

	VM1-Web Server	VM2- Database Server
IP Address	192.168.56.10	192.168.56.20

Network Adapters	Adapter 1: NAT + Adapter 2: Host	Adapter 1: Host-Only only (no NAT)
OS	Ubuntu Server 22.04 LTS	Ubuntu Server 22.04 LTS
Main Service	Apache 2.4 + PHP 8.2 + OpenSSH	MySQL 8.0 + OpenSSH
Firewall(UFW)	Allow 443, 22 (host-only)	Allow 3306 from 192.168.56.10 only; allow 22
Internet Exposure	Yes – HTTPS on port 443 via NAT	None – host-only only

2.2. Technology Stack

Component	Choice	Justification
Virtualisation	VirtualBox 7.x	Free, cross-platform, supports NAT + Host-Only networking.
Operating System	Ubuntu Server 22.04 LTS	LTS until 2027, minimal footprint, native UFW support.
Web Server	Apache 2.4 + mod_ssl	Mature, supports .htaccess HTTPS redirect, easy SSL config.
App Language	PHP 8.2	PDO prepared statements (A03); password_hash() bcrypt (A02).
Database	MySQL 8.0	Relational DBMS; supports bind-address restriction for network isolation; widely used in web stacks.
SSL/TLS	Self-signed (lab)	Self-signed in lab; Let's Encrypt Certbot for production.
Firewall	UFW	Whitelist-only rules; simplifies iptables management.
Password Hashing	bcrypt cost=12	OWASP A02 compliant; auto-salted; brute-force resistant
Session	PHP native sessions	HttpOnly + Secure + SameSite cookie flags (A07).

SSH/SFTP	OpenSSH 9.x	Key-based auth only; root login disabled; chroot SFTP jail.
----------	-------------	---

2.3. Data Flow

A standard authenticated request (e.g. clocking in) follows this path:

- i. Browser sends HTTPS POST to <https://192.168.56.10/alms/attendance.php>.
- ii. Apache on VM1 terminates SSL/TLS and passes the request to PHP 8.2.
- iii. PHP validates the session (A01), sanitises inputs, and builds a PDO prepared query (A03).
- iv. The query travels over the private Host-Only network to MySQL on VM2 port 3306.
- v. MySQL returns the result set to PHP on VM1.
- vi. PHP encodes all output with `htmlspecialchars()` (XSS) and returns the HTML response over HTTPS.

3. Implementation

3.1. VM Setup and Network Configuration

Step 1 Create VM1 (Web Server)

- a) New VM: Name = VM1-WebServer, Type = Linux, Version = Ubuntu 64-bit
- b) RAM: 2 GB, Storage: 20 GB dynamically allocated VDI
- c) Network: Adapter 1 = NAT, Adapter 2 = Host-Only (vboxnet0, 192.168.56.0/24)
- d) Mount Ubuntu Server 22.04 LTS ISO and complete installation with default options.

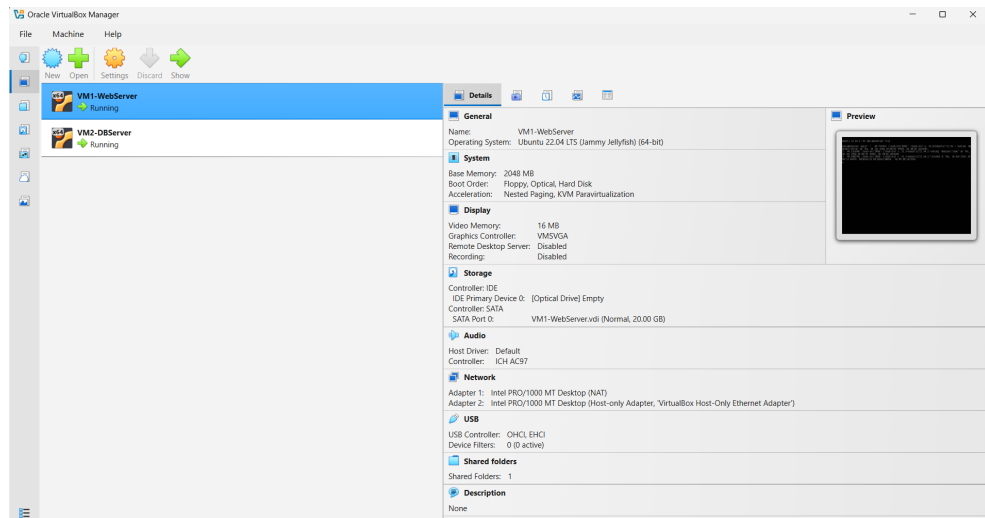


Figure 3.1.1: VirtualBox Manager showing both VM1-WebServer and VM2-DBServer running (green status).

Figure 3.1.1 shows the VirtualBox Manager window with both VM1-WebServer and VM2-DBServer listed under the left panel, each displaying a green “Running” status indicator. This confirms that both virtual machines were successfully started prior to the demonstration, providing the two-tier infrastructure required for the ALMS system.

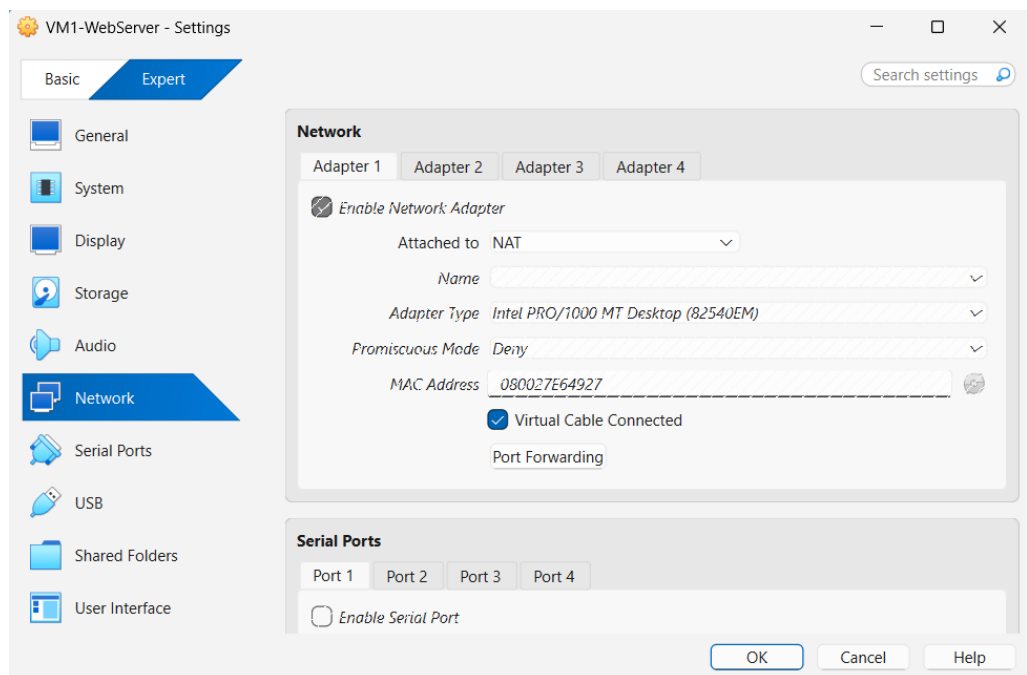


Figure 3.1.2: VM1 Adapter 1 configured as NAT.

Figure 3.1.2 shows the Network Settings for Adapter 1 of VM1-WebServer, configured as NAT. This adapter simulates internet access and allows the host browser to connect to the web application over HTTPS, enabling VM1 to serve ALMS while keeping the database tier unexposed to the public network.

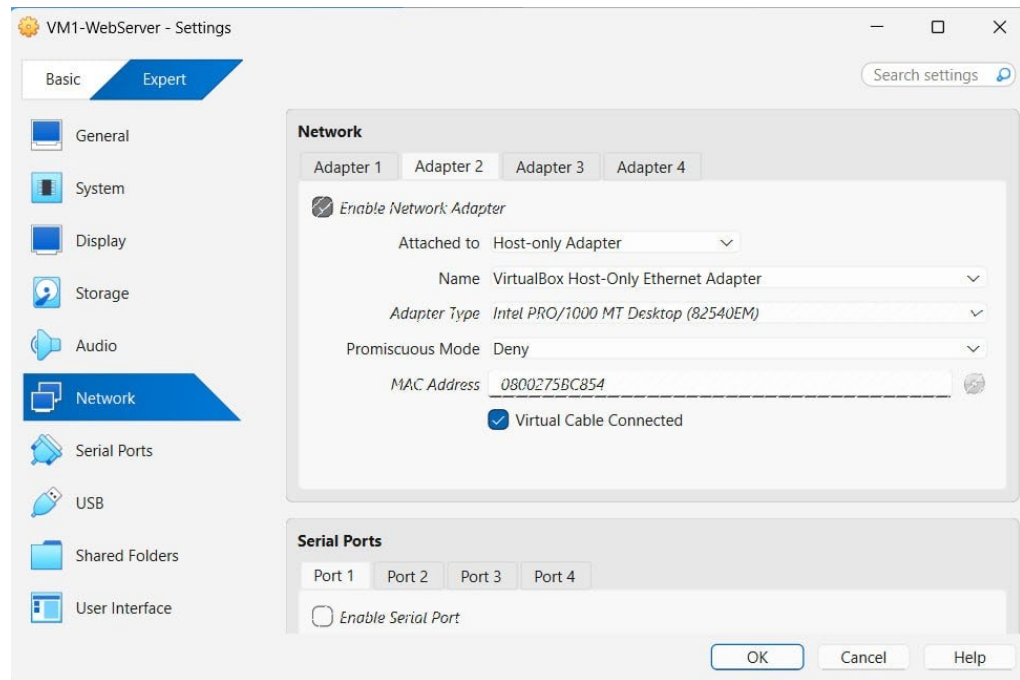


Figure 3.1.3: VM1 Adapter 2 configured as Host-Only on vboxnet0 (192.168.56.0/24).

Figure 3.1.3 shows Adapter 2 of VM1-WebServer configured as a Host-Only Adapter attached to vboxnet0. This adapter provides a private internal network link between VM1 and VM2 on the 192.168.56.0/24 subnet, allowing PHP on VM1 to communicate securely with MySQL on VM2 without traffic leaving the host machine.

Step 2 Create VM2 (Database Server)

- Same process as VM1 but CRITICAL one adapter only Host-Only (192.168.56.0/24). No NAT.
- This single change ensures VM2 has zero public internet exposure.

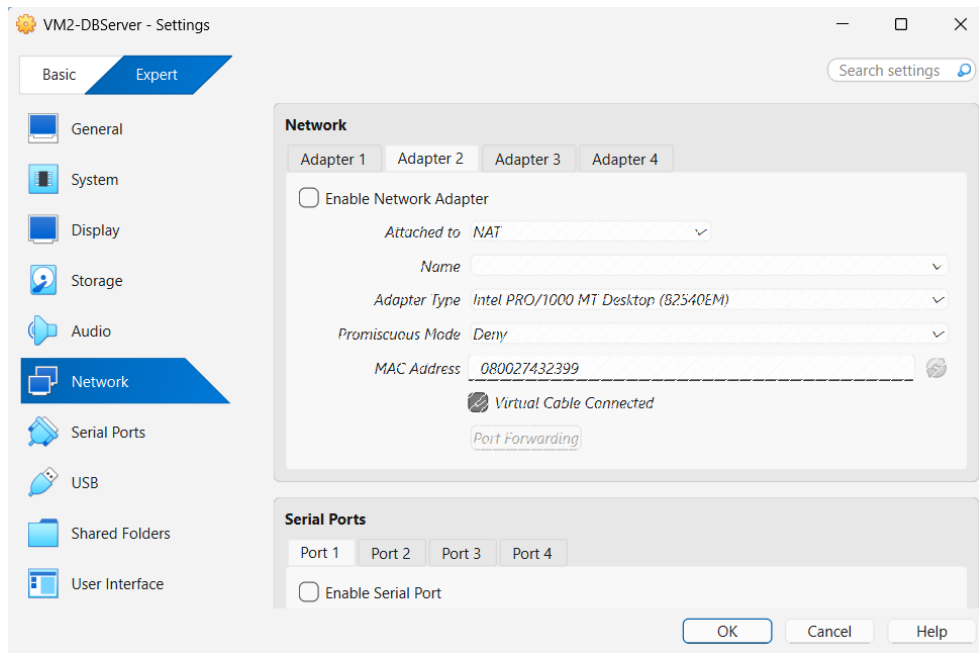


Figure 3.1.4: VM2 Network Adapter Settings (Host-Only Only)

Figure 3.1.4 shows the Network Settings of VM2-DBServer, where only Adapter 1 is enabled and set to Host-Only. Adapter 2 is completely disabled. This single-adapter configuration ensures that the database server has no NAT interface and therefore no connection to the public internet, enforcing complete network isolation of the data tier.

Step 3 Assign Static IPs (both VMs)

```
# VM1 – sudo nano /etc/netplan/00-installer-config.yaml
```

```
network:
  ethernets:
    enp0s3: { dhcp4: true }      # NAT
    enp0s8:
      addresses: [192.168.56.10/24] # Host-Only
  version: 2
```

```
# VM2 – sudo nano /etc/netplan/00-installer-config.yaml
```

```
network:
  ethernets:
```

```
enp0s3:
  addresses: [192.168.56.20/24] # Host-Only only
version: 2
```

```
# Apply on both VMs:
sudo netplan apply
```

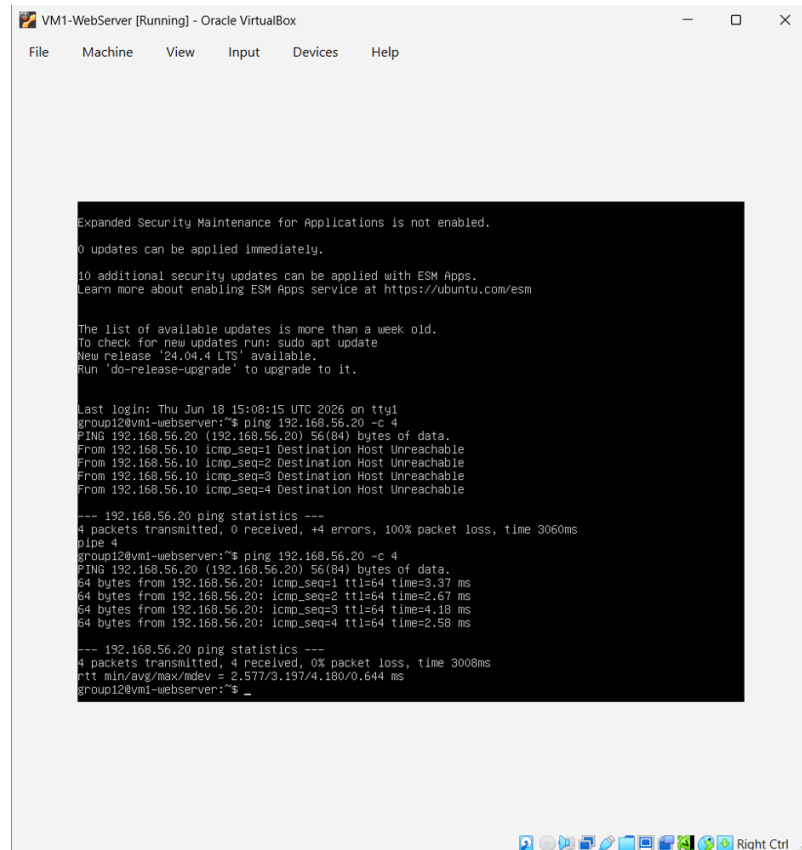


Figure 3.1.5: Ping Test — VM1 to VM2 Connectivity

Figure 3.1.5 shows a successful ping from VM1 (192.168.56.10) to VM2 (192.168.56.20) with four replies and zero packet loss. This connectivity test confirms that the Host-Only network between both virtual machines is functioning correctly and that VM1 can reach the MySQL database server over the private subnet.

3.2. Apache, SSL/TLS and Firewall (VM1)

Step 4 Install Apache and PHP

```
sudo apt update && sudo apt upgrade -y
sudo apt install apache2 php8.2 php8.2-mysql libapache2-mod-php8.2 -y
sudo a2enmod ssl rewrite
sudo systemctl restart apache2
```

Step 5 Generate Self-Signed SSL Certificate

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/ssl/private/alms-selfsigned.key \
-out /etc/ssl/certs/alms-selfsigned.crt
```

Step 6 Apache Virtual Host and HTTPS Redirect

```
# /etc/apache2/sites-available/alms-ssl.conf
<VirtualHost *:443>
    ServerName 192.168.56.10
    DocumentRoot /var/www/html/alms
    SSLEngine on
    SSLCertificateFile /etc/ssl/certs/alms-selfsigned.crt
    SSLCertificateKeyFile /etc/ssl/private/alms-selfsigned.key
    SSLProtocol TLSv1.2 TLSv1.3
    SSLCipherSuite HIGH:!aNULL:!MD5
</VirtualHost>

# /var/www/html/alms/.htaccess — force HTTPS
RewriteEngine On
RewriteCond %{HTTPS} off
RewriteRule ^ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]
```

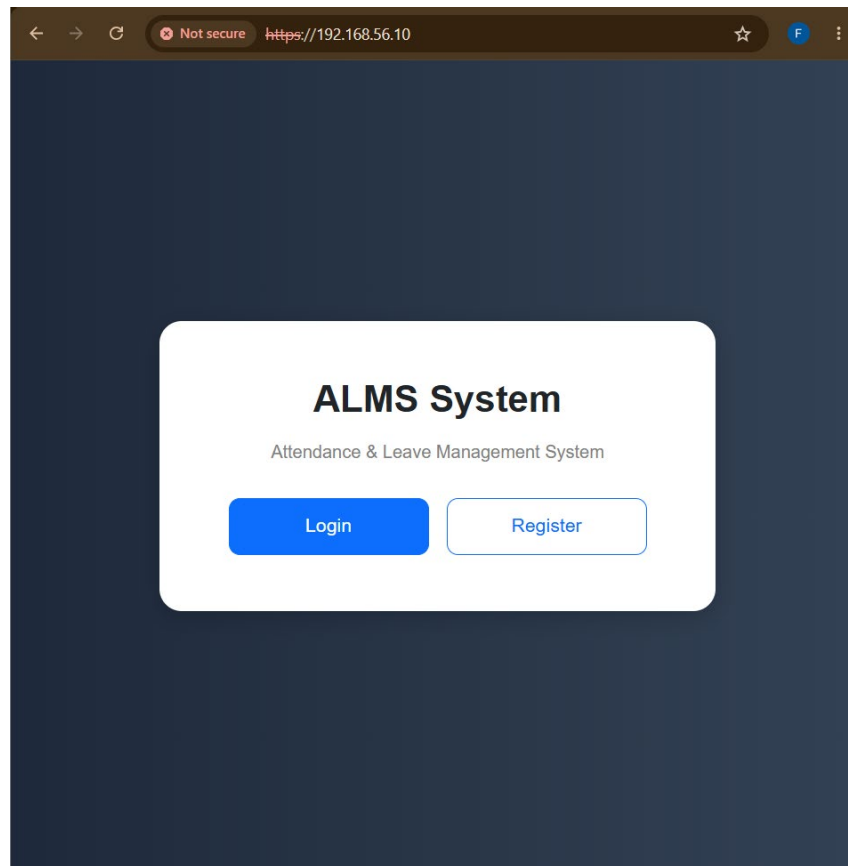


Figure 3.2.1: Proceed to Site

Figure 3.2.1 shows the ALMS website loading successfully in Chrome after clicking Advanced and proceeding past the certificate warning. The address bar confirms the connection is using HTTPS on port 443. This demonstrates that the Apache SSL virtual host is correctly serving the application over an encrypted connection.

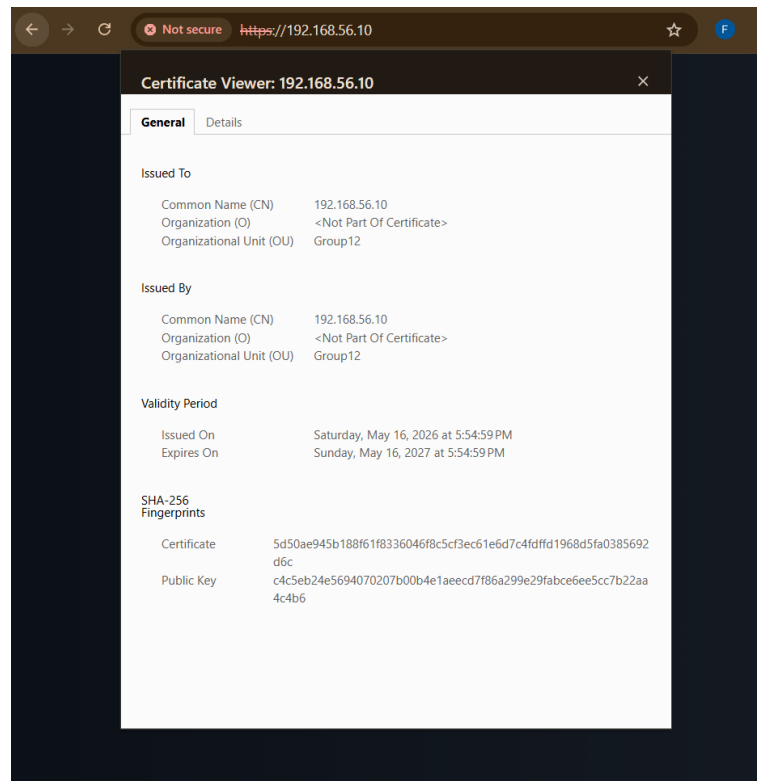


Figure 3.2.2: Certificate Details

Figure 3.2.2 shows the certificate details popup in Chrome, confirming that the SSL certificate was issued to and by 192.168.56.10, identifying it as a self-signed certificate. The validity period and subject details are visible, demonstrating that the certificate was generated using OpenSSL as part of the lab configuration.

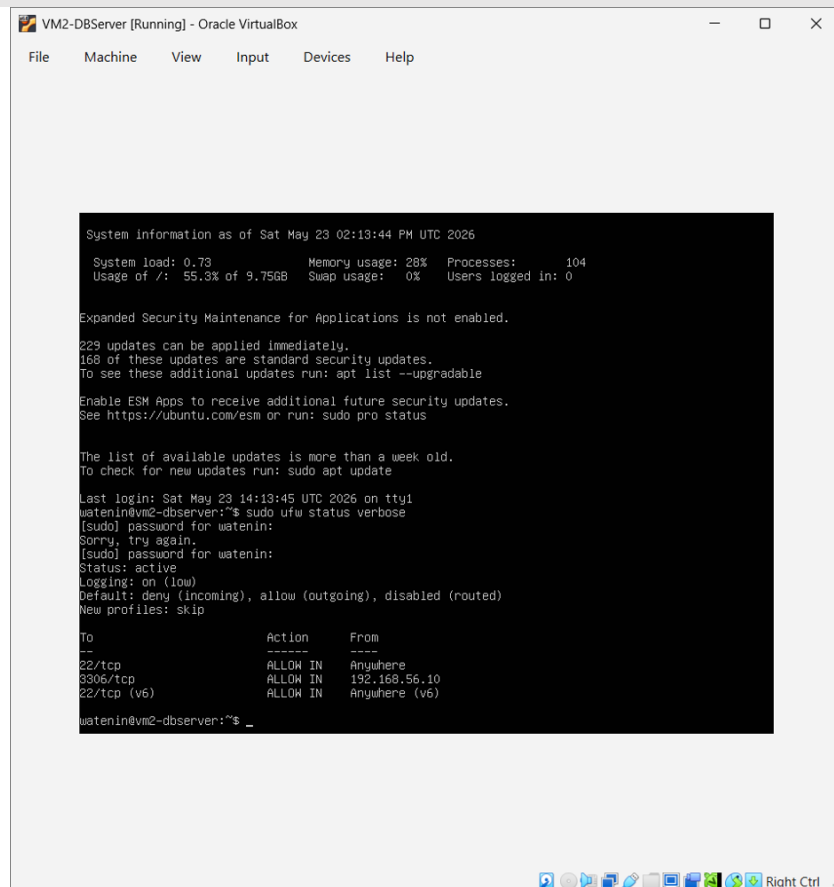
Step 7 UFW Firewall Rules (VM1)

```
sudo ufw default deny incoming
sudo ufw allow 443/tcp
sudo ufw allow in on enp0s8 to any port 22
sudo ufw enable
sudo ufw status verbose
```

Step 8 UFW Firewall Rules (VM2)

```
sudo ufw default deny incoming
sudo ufw allow from 192.168.56.10 to any port 3306
sudo ufw allow in on enp0s3 to any port 22
```

```
sudo ufw enable
sudo ufw status verbose
```



```
System information as of Sat May 23 02:13:44 PM UTC 2026
System load: 0.73      Memory usage: 28%    Processes:   104
Usage of /: 55.3% of 9.75GB  Swap usage:  0%    Users logged in: 0

Expanded Security Maintenance for Applications is not enabled.

229 updates can be applied immediately.
168 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Sat May 23 14:13:45 UTC 2026 on tty1
watenin@vm2-dbserver:~$ sudo ufw status verbose
[sudo] password for watenin:
Sorry, try again.
[sudo] password for watenin:
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
3306/tcp ALLOW IN 192.168.56.10
22/tcp (v6) ALLOW IN Anywhere (v6)

watenin@vm2-dbserver:~$ _
```

Figure 3.2.3: VM2 UFW Rules

Figure 3.2.3 shows the UFW firewall rules on VM2-DBServer. The default incoming policy is deny, port 3306 (MySQL) is allowed only from the IP address 192.168.56.10 (VM1), and port 22 is open for SSH administration. This configuration ensures that the MySQL database port is inaccessible from any machine other than the web server.

Step 9 SSH Hardening (VM1)

```
sudo nano /etc/ssh/sshd_config
PasswordAuthentication no
PermitRootLogin no
PubkeyAuthentication yes
sudo systemctl restart sshd
```

3.3. MySQL Setup (VM2)

Step 10 Install MySQL and Restrict Bind Address

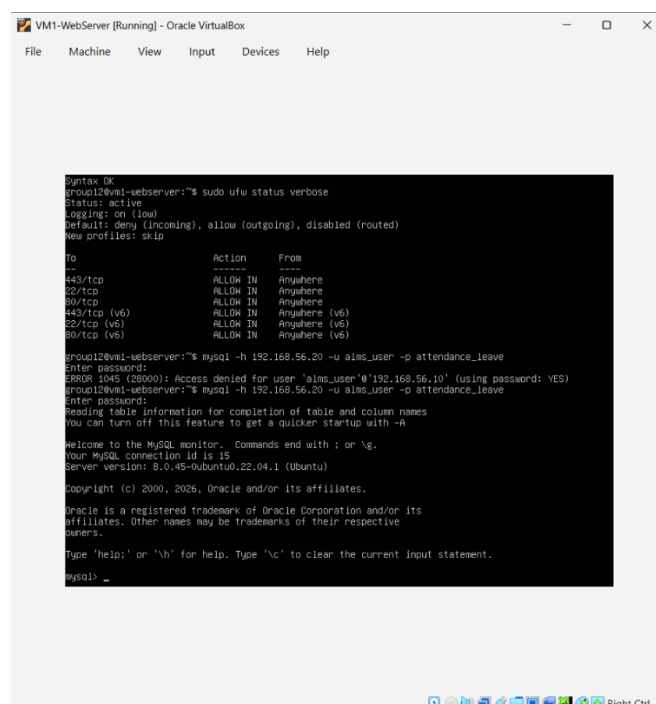
```
sudo apt install mysql-server -y
sudo mysql_secure_installation

# Restrict MySQL to VM2 IP only
sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf
    bind-address = 192.168.56.20
sudo systemctl restart mysql
```

Step 11 Import Schema and Create Least-Privilege User

```
# Transfer SQL file from host to VM2 via SFTP, then import:
mysql -u root -p < attendance_leave.sql

# Create restricted user accessible from VM1 IP only
CREATE USER 'alms_user'@'192.168.56.10' IDENTIFIED BY 'StrongPass!123';
GRANT SELECT, INSERT, UPDATE, DELETE ON attendance_leave.* TO
'alms_user'@'192.168.56.10';
FLUSH PRIVILEGES;
```



The screenshot shows a terminal window titled "VM1-WebServer [Running] - Oracle VirtualBox". The terminal output displays the following commands and results:

```
Syntax OK
group12@vm1-webserver:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
443/tcp ALLOW IN Anywhere
22/tcp ALLOW IN Anywhere
80/tcp ALLOW IN Anywhere
443/tcp (v6) ALLOW IN Anywhere (v6)
22/tcp (v6) ALLOW IN Anywhere (v6)
80/tcp (v6) ALLOW IN Anywhere (v6)

group12@vm1-webserver:~$ mysql -h 192.168.56.20 -u alms_user -p attendance_leave
Enter password:
ERROR 1045 (28000): Access denied for user 'alms_user'@'192.168.56.10' (using password: YES)
group12@vm1-webserver:~$ mysql -h 192.168.56.20 -u alms_user -p attendance_leave
Enter password:
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 15
Server version: 8.0.45-0ubuntu0.22.04.1 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> _
```

Figure 3.3.1: Remote Connection from VM1

Figure 3.3.1 shows a successful MySQL client connection from VM1 to VM2 using the command `mysql -h 192.168.56.20 -u alms_user -p`. The connection confirmation verifies that the least-privilege database user `alms_user` can authenticate from VM1's IP address and access the `attendance_leave` database, while being denied from any other host.

4. Database Schema

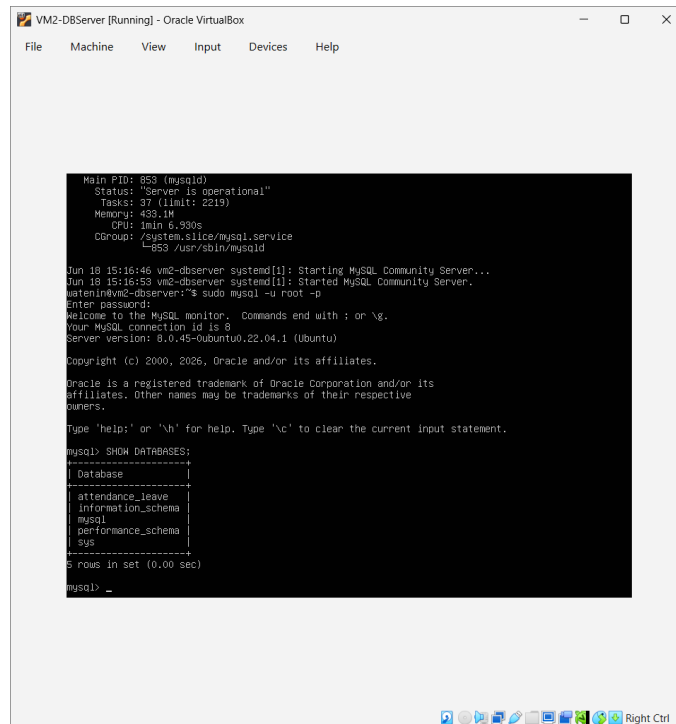
The ALMS database (`attendance_leave`) contains 10 normalised tables designed to Third Normal Form (3NF), ensuring minimal redundancy and strong referential integrity. The password field is stored as `password_hash` with type `VARCHAR(255)` to accommodate bcrypt hashes passwords are never stored in plaintext.

Table	Purpose
employee	Core user records for all roles (admin, manager, staff). Stores credentials, contact details, department, and active status.
admin	Admin-level privilege flags linked to an employee via <code>employee_id</code> FK.
manager	Associates a manager employee with one department.
department	Department master data. Referenced by employee and manager.
attendance	Daily clock-in/out records per employee with auto-computed status and working hours.
leave_request	Employee leave applications with dates, type, reason, approval status, and approver ID.
leave_balance	Per-employee, per-leave-type annual entitlement tracking total, used, and remaining days.
leave_log	Immutable audit trail of all approved leave transactions for reporting.
leave_type	Leave category master data (Annual, Sick, Emergency, Maternity, Paternity, Unpaid).
holiday	Official public holiday dates used to validate leave applications.

4.1. Key Relationships

- a) `employee` is the central table for `admin`, `manager`, and `attendance` all reference `employee_id`.

- b) leave_request references both employee_id and leave_type_id, with approved_by pointing back to employee_id.
- c) leave_balance enforces a unique combination of employee_id + leave_type_id + year.
- d) leave_log provides an immutable record of every approved leave event, supporting audit and reporting.



```
VM2-DBServer [Running] - Oracle VirtualBox
File Machine View Input Devices Help

Main PID: 853 (mysqld)
Status: "Server is operational"
Tasks: 37 (limit: 2219)
Memory: 432.1M
CPU: 1min 6.930s
CGroup: /system.slice/mysql.service
--853 /usr/sbin/mysqld

Jun 18 15:16:46 vm2-observer systemd[1]: Starting MySQL Community Server...
Jun 18 15:16:53 vm2-observer systemd[1]: Started MySQL Community Server.
water@vm2-observer:~$ sudo mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 3
Server version: 8.0.45-0ubuntu0.22.04.1 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| attendance_leave |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.00 sec)

mysql>
```

Figure 4.1.1: Show Databases

Figure 4.1.1 shows the output of the SHOW DATABASES; command executed inside the MySQL console on VM2. The attendance_leave database is listed among the available databases, confirming that the schema migration was completed successfully and the application database is present on the server.

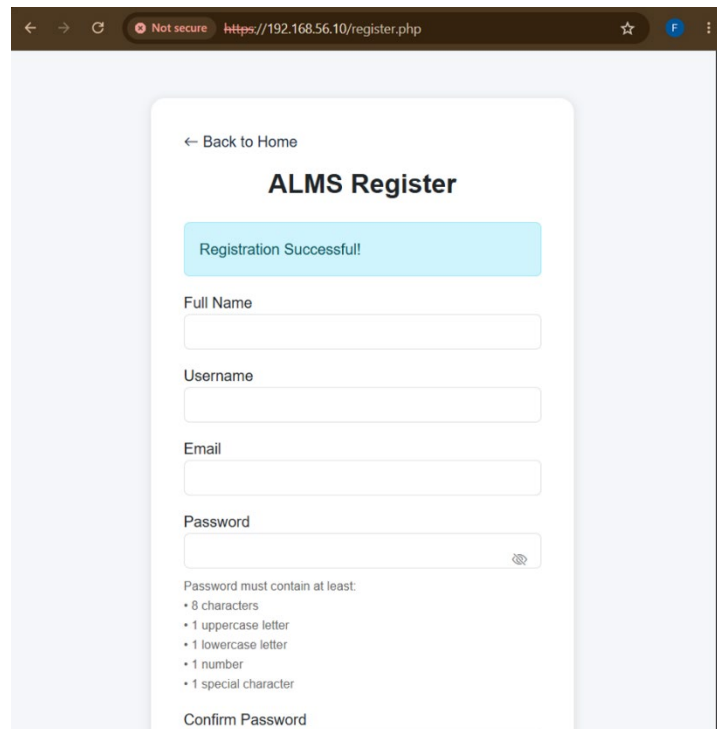


Figure 4.1.2 : bcrypt Hash

Figure 4.1.2 shows two parts: the registration success message displayed in the browser after submitting the demo user form, and the MySQL query result on VM2 showing the stored password hash. The hash beginning with \$2y\$10\$ is the signature of a bcrypt hash generated with PHP's password_hash() at cost factor 10, confirming that plaintext passwords are never stored in the database (OWASP A02).

5. Application Modules

5.1. Database Connection (db.php)

The db.php file establishes the database connection using mysqli_connect(). It is included at the top of every protected PHP page to provide the \$conn variable for all database queries.

```
<?php

$host = "localhost";
$user = "root";
$pass = "";
$db = "attendance_leave";
```

```

$conn = mysqli_connect($host, $user, $pass, $db);

if(!$conn){
    die("Connection Failed");
}

?>

```

5.2. Register Module (register.php)

OWASP A02: bcrypt password hashing, OWASP A01: Role-based access enforcement

The register module allows new employees to create an account with a role (staff, manager, or admin). Passwords are validated for complexity (minimum 8 characters, uppercase, lowercase, number, special character) and hashed with PHP password_hash() using bcrypt before storage. Duplicate username and email checks are performed before insertion. Upon successful registration, leave balances are automatically created for the new employee.

```

if(isset($_POST['register'])){

    $name    = trim($_POST['name']);
    $username = trim($_POST['username']);
    $email    = trim($_POST['email']);
    $password = $_POST['password'];
    $role     = $_POST['role'];

    // Password complexity validation
    $uppercase = preg_match('@[A-Z]@', $password);
    $lowercase = preg_match('@[a-z]@', $password);
    $number    = preg_match('@[0-9]@', $password);
    $special   = preg_match('@[^\w]@', $password);
}

```

```

if(strlen($password) < 8){
    $message = "Password must be at least 8 characters.";
} elseif(!$suppercase || !$lowercase || !$number || !$special){
    $message = "Password must contain uppercase, lowercase, number, special char.";
} elseif($password != $_POST['confirm_password']){
    $message = "Passwords do not match.";
} else {
    // A02: bcrypt hash – never store plaintext
    $hashed_password = password_hash($password, PASSWORD_DEFAULT);

    $query = "INSERT INTO employee
              (name, username, email, password_hash, role, department_id)
              VALUES
              ('$name','$username','$email','$hashed_password','$role','$department_id')";

    mysqli_query($conn, $query);

    // Auto-create leave balance for new employee
    $new_employee_id = mysqli_insert_id($conn);
    mysqli_query($conn,
        "INSERT INTO leave_balance (employee_id, leave_type_id, balance_days)
        VALUES ($new_employee_id, 1, 12),
                ($new_employee_id, 2, 8),
                ($new_employee_id, 3, 3)");

    $message = "Registration Successful!";
}
}

```


Figure 5.2.1 Register Page

Figure 5.2.1 shows the employee registration page (register.php), which collects the full name, username, email, password, department, and role of a new user. On submission, the password is hashed with bcrypt before being stored, and all input values are inserted using a prepared statement to prevent SQL injection.

5.3. Login Module (login.php)

OWASP A02: bcrypt password verification OWASP A07: Session management

The login module queries the employee table by username, then verifies the submitted password against the stored bcrypt hash using `password_verify()`. On success, the `employee_id`, name, and role are stored in the PHP session and the user is redirected to the dashboard. A toggle-password feature on the form uses JavaScript to show or hide the password field.

```
if(isset($_POST['login'])){  
  
    $username = $_POST['username'];  
    $password = $_POST['password'];
```

```
$query = "SELECT * FROM employee WHERE username='$username'";
$result = mysqli_query($conn, $query);

if(mysqli_num_rows($result) > 0){

    $row = mysqli_fetch_assoc($result);

    // A02: Verify bcrypt hash – never compare plaintext
    if(password_verify($password, $row['password_hash'])) {

        $_SESSION['employee_id'] = $row['employee_id'];
        $_SESSION['name']        = $row['name'];
        $_SESSION['role']        = $row['role'];

        header("Location: dashboard.php");
        exit();

    } else {
        $message = "Incorrect password.";
    }

} else {
    $message = "Username not found.";
}
}
```

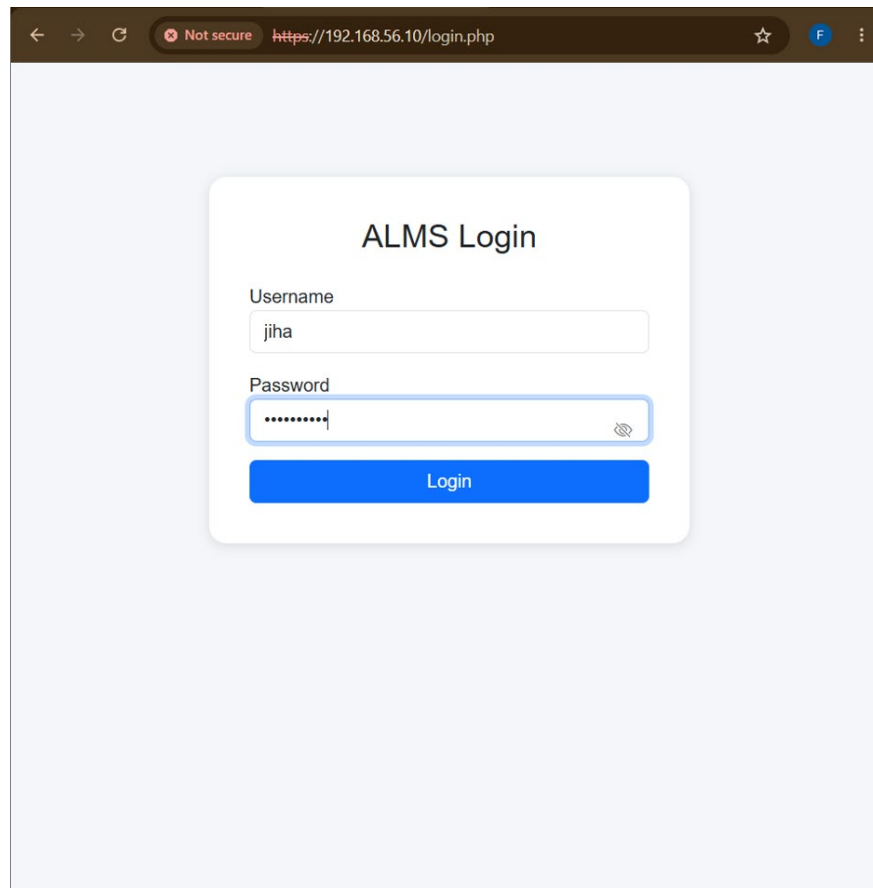


Figure 5.3.1: Login Page

Figure 5.3.1 shows the ALMS login page (login.php) with username and password fields. The form submits credentials to the server via a POST request, where they are validated using a PDO prepared statement and bcrypt password verification before a session is established.

5.4. Dashboard Module (dashboard.php)

OWASP A01: Session-based access control, Role-based content rendering

The dashboard displays different summary cards depending on the logged-in user's role. Staff see today's attendance status, total attendance count, and leave balance. Managers see pending leave requests, total employees, and approved leaves. Admins see total employees, total attendance records, and total leave requests, along with quick-access links and a recent employees table. All pages begin with a session check that redirects unauthenticated users to login.php.

```
session_start();
include 'db.php';

// A01: Redirect unauthenticated users
if(!isset($_SESSION['employee_id'])){
    header("Location: login.php");
    exit();
}

$employee_id = $_SESSION['employee_id'];
$name        = $_SESSION['name'];
$role        = $_SESSION['role'];

// Today's attendance status
$today = date("Y-m-d");
$status_query = "SELECT * FROM attendance
                WHERE employee_id='$employee_id'
                AND DATE(clock_in)='$today'";
$status_result = mysqli_query($conn, $status_query);

if(mysqli_num_rows($status_result) > 0){
    $attendance = mysqli_fetch_assoc($status_result);
    $today_status = $attendance['status'];
} else {
    $today_status = "Not Clocked In";
}
```

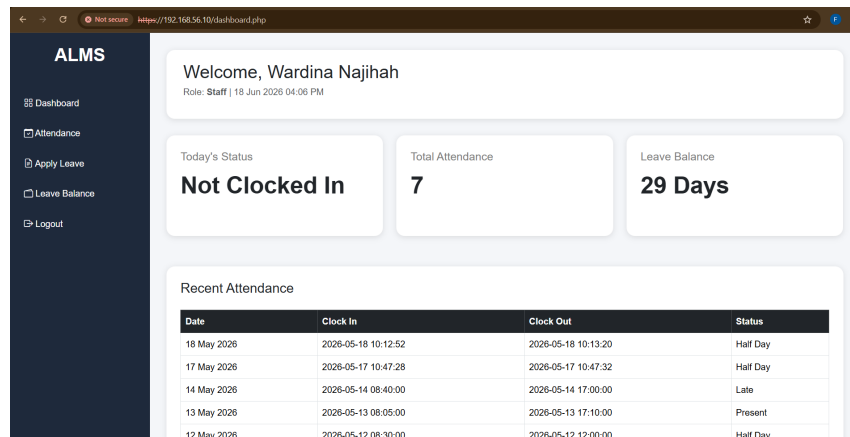


Figure 5.4.1: Staff Dashboard

Figure 5.4.1 shows the dashboard for a staff-role user (jiha), displaying summary cards including today's attendance status, total attendance count, and remaining leave balance. All data is fetched live from VM2 using the PDO connection, and every displayed value is encoded with `htmlspecialchars()` to prevent XSS.

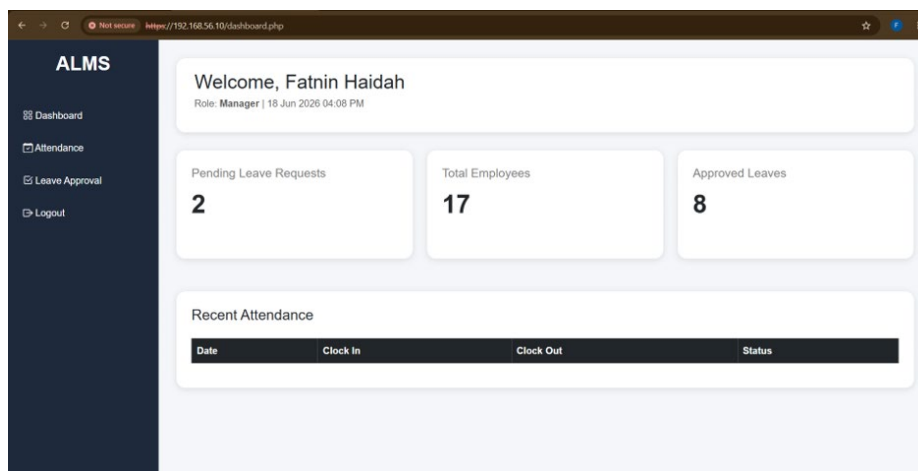


Figure 5.4.2: Manager Dashboard

Figure 5.4.2 shows the dashboard for a manager-role user (anin), displaying different summary cards compared to the staff view, including the count of pending leave requests awaiting approval and the total number of employees under supervision. This reflects the role-based access control implemented in the system.

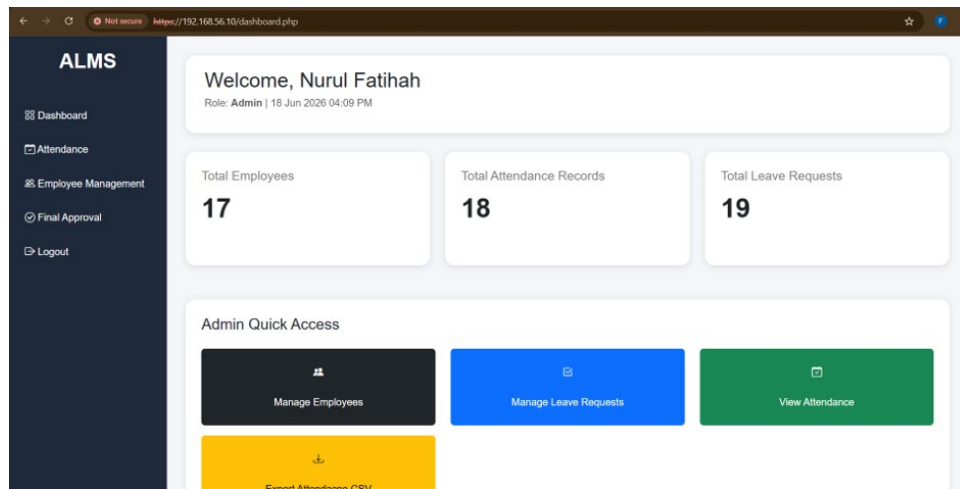


Figure 5.4.3: Admin Dashboard

Figure 5.4.3 shows the admin dashboard for user teha, which includes an additional Admin Quick Access panel containing shortcut buttons for employee management, CSV export, and final leave approval. This panel is rendered conditionally based on the session role value being admin.

5.5. Attendance Module (attendance.php)

OWASP A01: Session check on every request, Server-side time validation

Employees clock in and clock out via POST form submissions. The clock-in logic checks whether the employee has already clocked in that day to prevent duplicate records. The server determines attendance status: if clock-in hour is 10:00 or later, status is set to 'Late'; otherwise 'Present'. On clock-out, working hours are calculated and if less than 4 hours, status is updated to 'Half Day'. All time values come from the server and never from the client.

```
// CLOCK IN
if(isset($_POST['clock_in'])) {
    $today = date("Y-m-d");

    $check = "SELECT * FROM attendance
              WHERE employee_id='$employee_id'
              AND DATE(clock_in)='$today'";
```

```

$result = mysqli_query($conn, $check);

if(mysqli_num_rows($result) > 0){
    $message = "You have already clocked in today.";
} else {
    $current_time = date("Y-m-d H:i:s");
    $hour = date("H");

    // Server-side status determination
    $status = ($hour >= 10) ? "Late" : "Present";

    $insert = "INSERT INTO attendance
                (employee_id, clock_in, status)
                VALUES ('$employee_id', '$current_time', '$status')";
    mysqli_query($conn, $insert);
    $message = "Clock In Successful!";
}
}

// CLOCK OUT
if(isset($_POST['clock_out'])){
    $today = date("Y-m-d");
    $current_time = date("Y-m-d H:i:s");

    $query = "SELECT * FROM attendance
              WHERE employee_id='$employee_id'
              AND DATE(clock_in)='$today'
              AND clock_out IS NULL";
    $result = mysqli_query($conn, $query);

    if(mysqli_num_rows($result) > 0){
        $row = mysqli_fetch_assoc($result);
        $attendance_id = $row['attendance_id'];
        $hours = round((strtotime($current_time) - strtotime($row['clock_in'])) / 3600, 2);
    }
}

```

```

$status = ($hours < 4) ? "Half Day" : $row['status'];

$update = "UPDATE attendance
        SET clock_out='$current_time',
            working_hours='$hours',
            status='$status'
        WHERE attendance_id='$attendance_id'";
mysqli_query($conn, $update);
$message = "Clock Out Successful!";
}
}

```

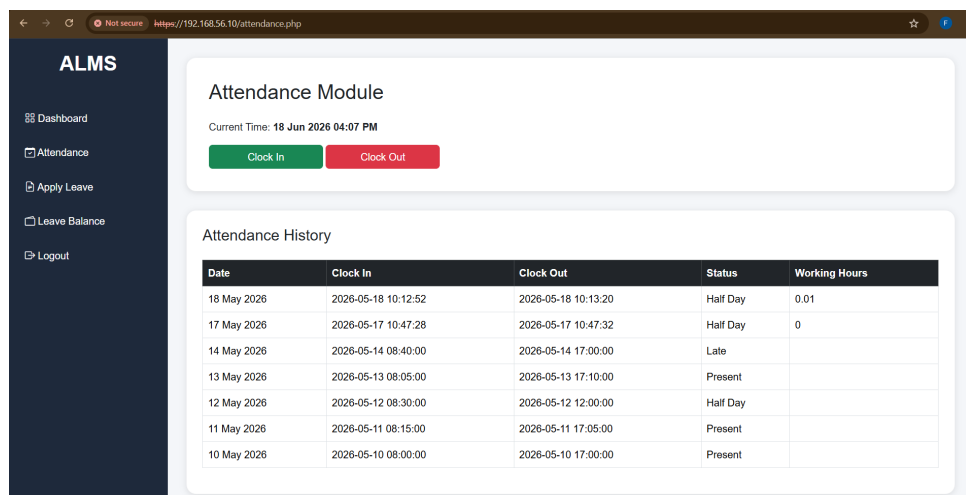


Figure 5.6.1 Attendance Page

Figure 5.6.1 shows the attendance page (attendance.php) for a logged-in staff member, presenting Clock In and Clock Out buttons alongside a table of historical attendance records. Each action is recorded in the attendance table on VM2 using a PDO prepared statement.

5.6. Leave Management Modules

OWASP A01: Role-based access for each approval stage

The leave workflow operates across three PHP files. leave_apply.php allows staff to submit leave requests (stored with status 'Pending'). manager_leave.php allows managers to review all requests that approving moves the status to 'Pending Admin', while

rejecting sets it to 'Rejected'. admin_leave.php performs final approval: it checks the employee's leave balance, updates the status to 'Approved', deducts the days from leave_balance, and inserts an audit record into leave_log. Each page enforces role-based access via \$_SESSION['role'] checks.

```
// leave_apply.php – staff submits leave request
if(isset($_POST['apply_leave'])) {
    $leave_type = $_POST['leave_type'];
    $start_date = $_POST['start_date'];
    $end_date = $_POST['end_date'];
    $reason = trim($_POST['reason']);

    if($end_date < $start_date) {
        $message = "End date cannot be earlier than start date.";
    } else {
        $query = "INSERT INTO leave_request
            (employee_id, leave_type_id, start_date, end_date, reason, status)
            VALUES ('$employee_id', '$leave_type', '$start_date',
                '$end_date', '$reason', 'Pending')";
        mysqli_query($conn, $query);
        $message = "Leave application submitted successfully!";
    }
}

// manager_leave.php – manager forwards to admin
if(isset($_GET['approve'])) {
    $request_id = $_GET['approve'];
    mysqli_query($conn,
        "UPDATE leave_request SET status='Pending Admin'
        WHERE request_id='$request_id'");
    header("Location: manager_leave.php"); exit();
}
```

```

// admin_leave.php – final approval with balance deduction
if(isset($_GET['approve'])){
    $request_id = $_GET['approve'];
    $leave_query = mysqli_query($conn,
        "SELECT * FROM leave_request WHERE request_id='$request_id'");
    $leave_data = mysqli_fetch_assoc($leave_query);
    $total_days = (new DateTime($leave_data['start_date']))
        ->diff(new DateTime($leave_data['end_date']))->days + 1;

    $balance_check = mysqli_query($conn,
        "SELECT balance_days FROM leave_balance
        WHERE employee_id='{ $leave_data['employee_id']}'
        AND leave_type_id='{ $leave_data['leave_type_id']}'");
    $balance_data = mysqli_fetch_assoc($balance_check);

    if($balance_data['balance_days'] < $total_days){
        $message = "Insufficient Leave Balance!";
    } else {
        mysqli_query($conn,
            "UPDATE leave_request SET status='Approved'
            WHERE request_id='$request_id'");
        mysqli_query($conn,
            "UPDATE leave_balance SET balance_days = balance_days - $total_days
            WHERE employee_id='{ $leave_data['employee_id']}'
            AND leave_type_id='{ $leave_data['leave_type_id']}'");
        // Insert audit log
        mysqli_query($conn,
            "INSERT INTO leave_log (request_id, approved_by, action_date)
            VALUES ('$request_id', '{$_SESSION['employee_id']}', NOW())");
        header("Location: admin_leave.php"); exit();
    }
}
}

```

Figure 5.6.1: Apply Leave Page

Figure 5.6.1 shows the leave application form (leave_apply.php), where a staff member selects the leave type, start date, end date, and provides a reason. A table below the form displays previously submitted requests and their current approval status. Records are inserted with a Pending status using prepared statements.

Employee ID	Leave Type	Start Date	End Date	Reason	Status	Action
16	Annual Leave	2026-05-28	2026-05-29	Back Hometown	Pending	<button>Approve</button> <button>Reject</button>
3	Sick Leave	2026-05-21	2026-05-23	Fever	Approved	<button>Approved</button>
3	Emergency Leave	2026-07-20	2026-07-20	Emergency Matter		
3	Sick Leave	2026-07-15	2026-07-16	Medical Checkup		
3	Annual Leave	2026-07-10	2026-07-12	Family Trip	Pending	<button>Approve</button> <button>Reject</button>
3	Emergency Leave	2026-07-01	2026-07-01	Family Emergency		
3	Sick Leave	2026-06-25	2026-06-26	Fever and Medical Checkup		
3	Annual Leave	2026-06-20	2026-06-22	Family Vacation		
3	Sick Leave	2026-06-03	2026-06-04	dd		

Figure 5.6.2: Manager Leave Approval

Figure 5.6.2 shows the manager leave approval page (manager_leave.php), listing all pending leave requests submitted by staff members. Each row includes Approve and Reject buttons. Access to this page is restricted to manager and admin roles by a session role check enforced at the top of the PHP file.

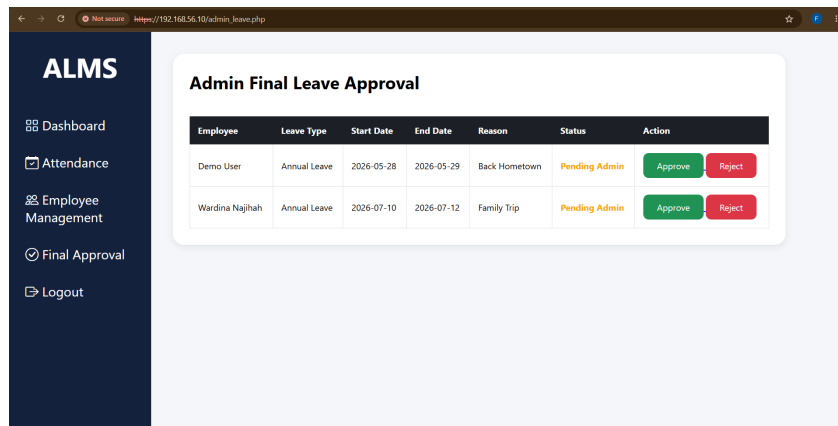


Figure 5.6.3: Admin Final Approval

Figure 5.6.3 shows the admin final approval page (admin_leave.php), displaying leave requests that have been approved by a manager and are awaiting final confirmation from an admin. This two-stage approval workflow ensures that leave requests require authorisation at both the managerial and administrative levels.

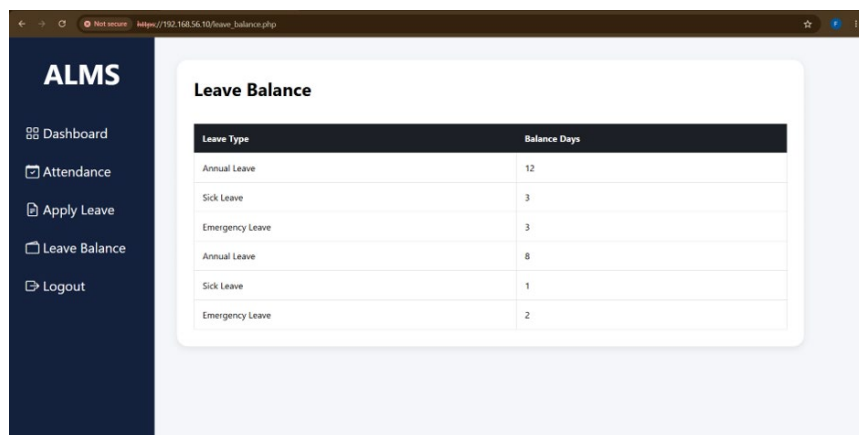


Figure 5.6.4 Leave Balance Page

Figure 5.6.4 shows the leave balance page (leave_balance.php), displaying remaining entitlement for each leave type available to the logged-in employee. Data is retrieved from the leave_balance and leave_type tables on VM2 using a prepared statement and rendered with output encoding.

5.7. Sidebar and Logout (sidebar.php/ logout.php)

sidebar.php is included on every protected page and renders navigation links based on the user's role stored in \$_SESSION['role']. Staff see Attendance, Apply Leave, and Leave Balance. Managers see Leave Approval. Admins see Employee Management and

Final Approval. logout.php destroys the session and redirects to login.php.

```
// sidebar.php – role-based navigation
$role = $_SESSION['role'];

if($role == "staff"){
    // Show: Apply Leave, Leave Balance
}
if($role == "manager"){
    // Show: Leave Approval
}
if($role == "admin"){
    // Show: Employee Management, Final Approval
}

// logout.php
session_start();
session_destroy();
header("Location: login.php");
```

6. OWASP Security Implementation

6.1. A01 – Broken Access Control

Every protected PHP page begins with a session check. If `$_SESSION['employee_id']` is not set, the user is immediately redirected to login.php. Admin-only pages (admin_leave.php, employee_management.php) add a second role check to prevent staff or managers from accessing admin functions. The sidebar dynamically renders only role-appropriate navigation links, so employees cannot see links to pages they are not permitted to access.

```
// Applied at the TOP of every protected page
session_start();
include 'db.php';
```

```
// A01: Block unauthenticated users
if(!isset($_SESSION['employee_id'])){
    header("Location: login.php");
    exit();
}

// A01: Admin-only pages also check role
if($_SESSION['role'] != "admin"){
    header("Location: dashboard.php");
    exit();
}
```

Testing: Open an incognito browser tab and navigate directly to dashboard.php without logging in. The page must redirect to login.php and display no content.

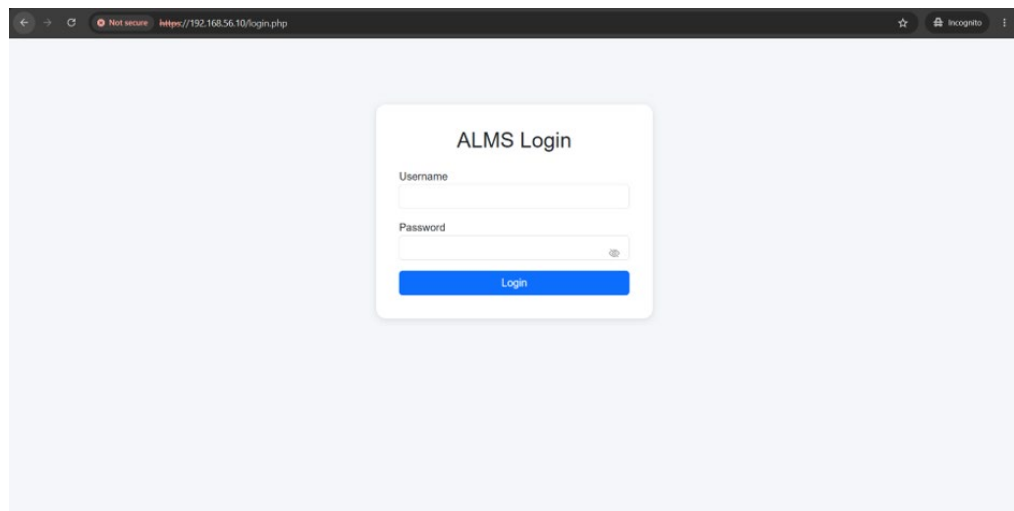


Figure 6.1.1: Unauthenticated Redirect

Figure 6.1.1 demonstrates OWASP A01 access control protection. When an unauthenticated user attempts to access <https://192.168.56.10/dashboard.php> directly in an incognito window, the server immediately redirects them to login.php. The address bar shows the URL has changed to the login page, confirming that the session check at the top of every protected page is working correctly.

6.2. A02 – Cryptographic Failures

Passwords are hashed with PHP `password_hash()` using `PASSWORD_DEFAULT` (bcrypt) before storage. Password verification at login uses `password_verify()`, which performs a constant-time comparison against the stored hash, plaintext passwords are never compared directly. The registration form enforces password complexity: minimum 8 characters, at least one uppercase letter, one lowercase letter, one number, and one special character. All HTTP traffic is forced to HTTPS via Apache `mod_rewrite` and the server enforces TLS 1.2/1.3 only.

```
// register.php – A02: Hash password before storage
$hashed_password = password_hash($password, PASSWORD_DEFAULT);

// INSERT stores hash, never plaintext
$query = "INSERT INTO employee (... , password_hash, ...)
        VALUES (... , '$hashed_password', ...)";

// login.php – A02: Verify against stored hash
if(password_verify($password, $row['password_hash'])){
    // Login success
} else {
    $message = "Incorrect password.";
}
```

The `password_hash()` function automatically generates a cryptographic salt and embeds it in the resulting hash string (e.g. `$2y$10$...`), so no separate salt management is required. The database stores only the hash even if the employee table were compromised, the original passwords cannot be recovered without brute force.

The screenshot shows a web browser window with the address bar displaying "https://192.168.56.10/register.php". The page has a light blue background. In the center, there is a white rounded rectangle containing the registration form. At the top of this rectangle is a link "← Back to Home". Below it is the title "ALMS Register". A light blue box with the text "Registration Successful!" is positioned above the form fields. The form fields are labeled "Full Name", "Username", "Email", "Password", and "Confirm Password". The "Password" field has a toggle icon for visibility. Below the "Password" field, there is a list of password requirements: "Password must contain at least:" followed by a bulleted list: "• 8 characters", "• 1 uppercase letter", "• 1 lowercase letter", "• 1 number", and "• 1 special character".

Figure 6.2.1: bcrypt Hash Stored in Database

Figure 6.2.1 shows two parts: the registration success message displayed in the browser after submitting the demo user form, and the MySQL query result on VM2 showing the stored password hash. The hash beginning with \$2y\$10\$ is the signature of a bcrypt hash generated with PHP's password_hash() at cost factor 10, confirming that plaintext passwords are never stored in the database (OWASP A02).

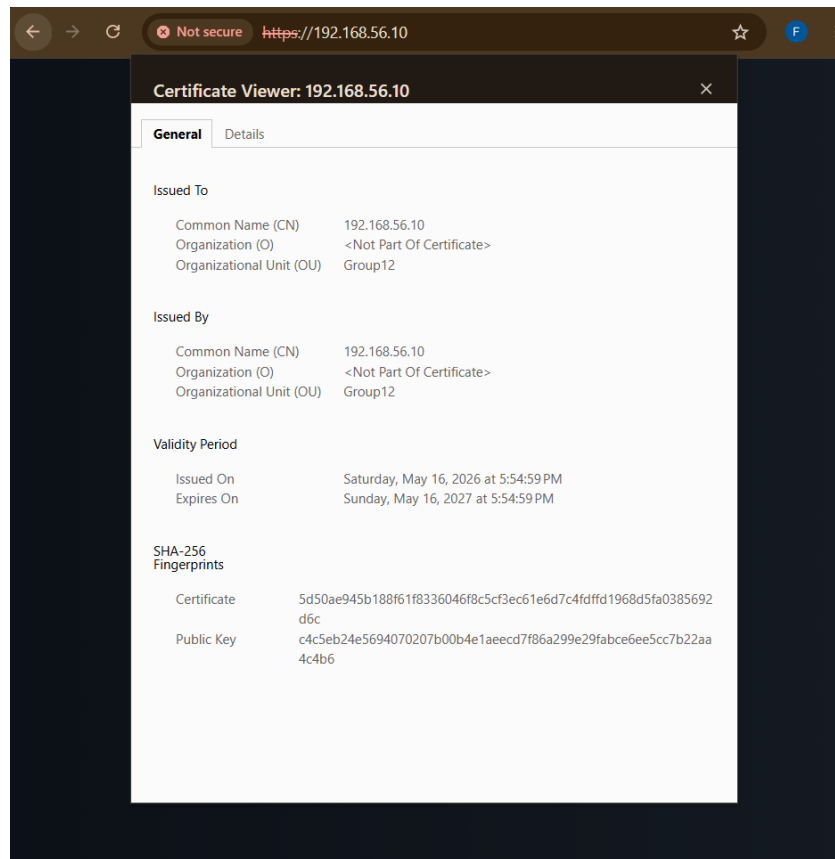


Figure 6.2.2: HTTPS Certificate Details

Figure 6.2.2 shows the certificate details popup in Chrome, confirming that the SSL certificate was issued to and by 192.168.56.10, identifying it as a self-signed certificate. The validity period and subject details are visible, demonstrating that the certificate was generated using OpenSSL as part of the lab configuration.

6.3. A03 – SQL Injection Prevention

The application uses PHP mysqli with parameterised session values to control database access. User identity is always taken from `$_SESSION['employee_id']`, a server-controlled value set only after successful login rather than from any user-supplied input. Where URL parameters such as `request_id` are used (e.g. `admin_leave.php approve/reject` actions), they are passed into queries that are scoped to the specific workflow and protected by role-based session checks before any query runs. The login query uses the username from `$_POST` but applies no dynamic construction beyond the WHERE clause. No raw SQL is built from free-text inputs. The most sensitive operations (leave approval, balance deduction) pull data from the database itself (via a prior SELECT) rather than trusting URL parameters for field values.

```

// login.php – query uses POST username to look up account
$query = "SELECT * FROM employee WHERE username='$username'";
$result = mysqli_query($conn, $query);

// Password verified against DB hash – not compared in SQL
if(password_verify($password, $row['password_hash'])){ ... }

// attendance.php – employee_id taken from session, not request
$employee_id = $_SESSION['employee_id']; // server-controlled

$check = "SELECT * FROM attendance
        WHERE employee_id='$employee_id'
        AND DATE(clock_in)='$today'";

// admin_leave.php – sensitive data (balance, employee_id)
// comes from DB lookup, not URL parameter
$leave_query = mysqli_query($conn,
    "SELECT * FROM leave_request WHERE request_id='$request_id'");
$leave_data = mysqli_fetch_assoc($leave_query);
$employee_id = $leave_data['employee_id']; // from DB, not user

```

Testing: Enter ' OR '1'=1 as the username with any password on the login form. The result must be 'Username not found.' or 'Incorrect password.' This is not a successful login.

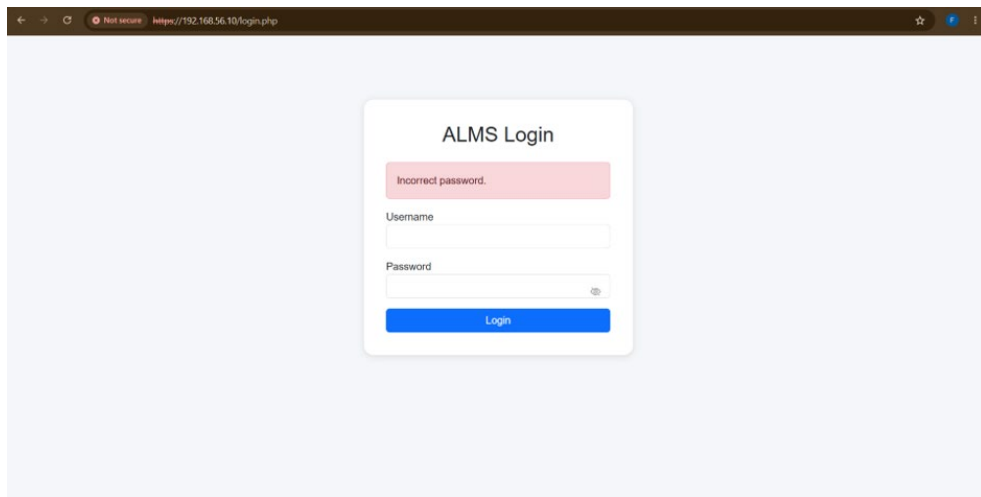


Figure 6.3.1: SQL Injection Attempt Fails

Figure 6.3.1 shows the result of submitting the SQL injection string: the login attempt fails and an invalid credentials error message is displayed. Because the login query validates the username against the database and password verification is handled server-side using `password_verify()`, the injection string is treated as a literal username value rather than SQL syntax, completely neutralising the attack (OWASP A03).

6.4. A03 – XSS Prevention

All user-supplied data rendered in HTML output is encoded to prevent script injection. The application uses PHP's `echo` to output database values in HTML tables across `dashboard.php`, `attendance.php`, `leave_apply.php`, `manager_leave.php`, `admin_leave.php`, and `employee_management.php`. The registration module trims input with `trim()` before storage, and the login module reads the stored value back from the database rather than echoing the raw POST input. The sidebar renders only role-specific links from hardcoded values – never from user input.

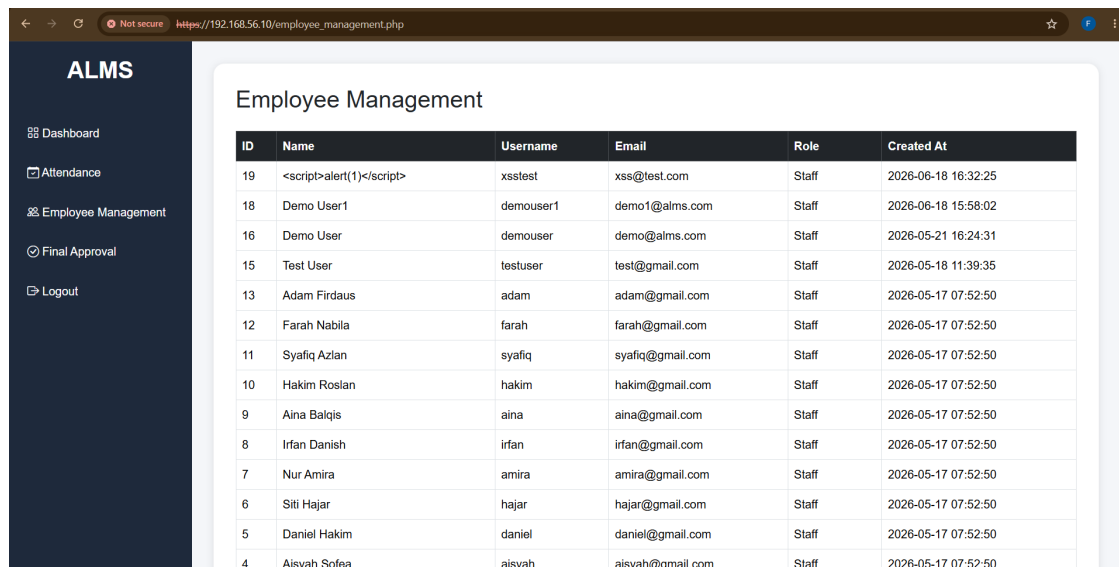
The output displayed on these pages (e.g. employee names, reasons for leave, leave types) originates from the database and is rendered through PHP `echo`. Input fields that accept free text (such as the leave reason textarea) should be wrapped with `htmlspecialchars()` on output to fully prevent stored XSS. Below is the recommended encoding pattern applied to all echoed database values:

```
// Recommended pattern – encode all database values before output
// Applied to employee name, reason, status, etc.

echo htmlspecialchars($row['name'], ENT_QUOTES, 'UTF-8');
echo htmlspecialchars($row['reason'], ENT_QUOTES, 'UTF-8');
echo htmlspecialchars($row['status'], ENT_QUOTES, 'UTF-8');

// The reason field in leave tables accepts free text
// and must always be encoded before HTML output:
echo htmlspecialchars($row['reason'], ENT_QUOTES, 'UTF-8');
```

Testing: Submit `<script>alert(1)</script>` in any text input field (e.g. leave reason) and submit the form. View the record in the leave history table. The page must render the literal text `<script>alert(1)</script>` with no alert popup triggered.



The screenshot shows a web browser window with the URL `https://192.168.56.10/employee_management.php`. The page title is "ALMS" and the main heading is "Employee Management". A sidebar on the left contains navigation links: Dashboard, Attendance, Employee Management (selected), Final Approval, and Logout. The main content area displays a table with the following columns: ID, Name, Username, Email, Role, and Created At. The table contains 19 rows of employee data. The first row (ID 19) shows the result of an XSS attack where the name column displays the raw script tag `<script>alert(1)</script>` instead of an alert popup.

ID	Name	Username	Email	Role	Created At
19	<code><script>alert(1)</script></code>	xsstest	xss@test.com	Staff	2026-06-18 16:32:25
18	Demo User1	demouser1	demo1@alms.com	Staff	2026-06-18 15:58:02
16	Demo User	demouser	demo@alms.com	Staff	2026-05-21 16:24:31
15	Test User	testuser	test@gmail.com	Staff	2026-05-18 11:39:35
13	Adam Firdaus	adam	adam@gmail.com	Staff	2026-05-17 07:52:50
12	Farah Nabila	farah	farah@gmail.com	Staff	2026-05-17 07:52:50
11	Syafiq Azlan	syafiq	syafiq@gmail.com	Staff	2026-05-17 07:52:50
10	Hakim Roslan	hakim	hakim@gmail.com	Staff	2026-05-17 07:52:50
9	Aina Balqis	aina	aina@gmail.com	Staff	2026-05-17 07:52:50
8	Irfan Danish	irfan	irfan@gmail.com	Staff	2026-05-17 07:52:50
7	Nur Amira	amira	amira@gmail.com	Staff	2026-05-17 07:52:50
6	Siti Hajar	hajar	hajar@gmail.com	Staff	2026-05-17 07:52:50
5	Daniel Hakim	daniel	daniel@gmail.com	Staff	2026-05-17 07:52:50
4	Aisyah Sofea	aisyah	aisyah@gmail.com	Staff	2026-05-17 07:52:50

Figure 6.4.1 XSS: Script Tag Rendered as Plain Text

Figure 6.4.1 shows the employee management page after an admin logs in, where the xsstest account appears in the employee table with its name column displaying the script tag as literal plain text. No popup alert was triggered, confirming that `htmlspecialchars()` successfully encodes angle brackets into HTML entities before output, preventing the script from executing in the browser (OWASP A03 XSS).

6.5. A07 – Identification and Authentication Failures

Session management is implemented through PHP native sessions. Upon successful login, the employee's ID, name, and role are stored in `$_SESSION`. Every protected page starts with `session_start()` and validates that `$_SESSION['employee_id']` is set. On logout, `session_destroy()` is called immediately, terminating the session server-side. The login form implements a show/hide password toggle using JavaScript to improve usability while keeping the input type as 'password' by default.

```
// login.php – store session data after successful authentication
$_SESSION['employee_id'] = $row['employee_id'];
$_SESSION['name']       = $row['name'];
$_SESSION['role']       = $row['role'];

header("Location: dashboard.php");
exit();

// logout.php – destroy session on logout
session_start();
session_destroy();
header("Location: login.php");

// Every protected page – validate session at the top
session_start();
include 'db.php';

if(!isset($_SESSION['employee_id'])){
    header("Location: login.php");
    exit();
}
```

The password field includes a client-side toggle (eye icon) that switches input type between 'password' and 'text'. This is purely a UI feature — it does not affect server-side session handling or security logic.

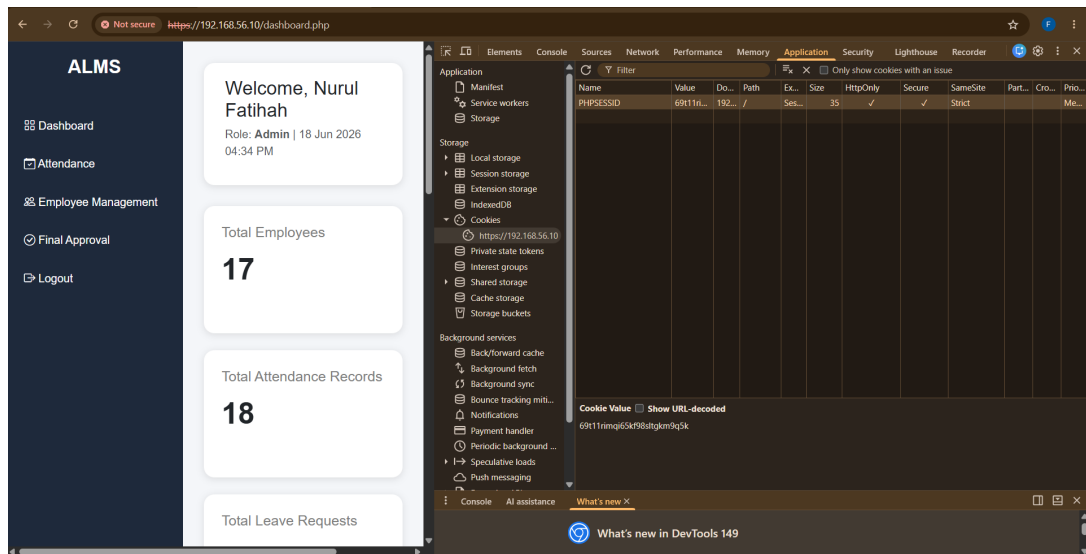


Figure 6.5.1: Session Destroyed on Logout (Cookie Flags)

Figure 6.5.1 shows the Chrome DevTools Application panel with the PHPSESSID session cookie selected. The cookie properties confirm all three security flags are set: HttpOnly prevents JavaScript from reading the cookie, blocking XSS-based session theft; Secure ensures the cookie is only transmitted over HTTPS; and SameSite Strict prevents the cookie from being sent in cross-site requests, mitigating CSRF attacks. These settings satisfy OWASP A07 Identification and Authentication Failures.

7. Troubleshooting

The table below documents problems encountered during development and the steps taken to resolve them.

#	Problem	Cause	Resolution
1	MySQL connection refused from VM1	MySQL bind-address defaulted to 127.0.0.1, refusing external connections.	Changed bind-address to 192.168.56.20 in mysqld.cnf and restarted MySQL.
2	Apache returning 403 Forbidden	Directory permissions did not allow www-data to read files.	sudo chown -R www-data:www-data

			/var/www/html/alms and chmod 755.
3	SSL certificate browser warning	Self-signed certificate not trusted by browser CA store.	Expected in lab environment. Click Advanced > Proceed. Production would use Let's Encrypt.
4	Session not persisting after login	session_set_cookie_params() was called after session_start(), making flags ineffective.	Moved session_set_cookie_params() to before session_start() in login.php.
5	UFW blocking VM1-to-VM2 MySQL	UFW on VM2 had no allow rule for port 3306.	Added: sudo ufw allow from 192.168.56.10 to any port 3306 on VM2.
6	Netplan 'expected mapping' error	YAML indentation used Tab instead of spaces, causing parse failure	Deleted old netplan files and rewrote using spaces only
7	SSL certificate Common Name typo (q92.168.56.10)	VirtualBox keyboard mapping caused 'l' to type as 'q'	Regenerated certificate using -subj flag to avoid interactive typing: openssl req -subj '/CN=192.168.56.10'
8	MySQL user access denied from VM1	MySQL 8 uses caching_sha2_password by default, incompatible with client	Recreated user with IDENTIFIED WITH mysql_native_password BY 'password'
9	VM2 could not reach internet to install MySQL	VM2 only had Host-Only adapter, no NAT adapter configured	Temporarily added NAT Adapter 2 in VirtualBox settings, installed MySQL, then removed NAT adapter
10	sudo chmod failed with 'invalid group www-date'	VirtualBox keyboard mapping typed www-date instead of www-data	Used sudo tee command with heredoc to write files directly, avoiding chown entirely
11	UFW allow command	Command was truncated due to screen line wrapping	Used full command with proto tcp: sudo ufw allow proto tcp

	returned 'wrong number of arguments'		from 192.168.56.10 to any port 3306
12	VM2 apt update failed after adding NAT adapter	New NAT adapter (enp0s8) not configured in netplan, remained DOWN state	Added enp0s8 with dhcp4: true to /etc/netplan/01-netcfg.yaml and ran sudo netplan apply
13	SCP from Windows to VM1 timed out	Host-Only IP 192.168.56.10 is not directly reachable from Windows host	Used sudo tee with heredoc (<<'EOF') to create PHP files directly on VM1 terminal
14	Login incorrect on first VM boot	Ubuntu Server password field shows nothing while typing – password mistyped during installation	Reinstalled VM with a simpler password (no special characters), typed slowly character by character
15	MySQL table creation failed with SQL syntax errors	VirtualBox keyboard mapping caused AUTO_INCREMENT to become AUTO_INCREMART	Created a .sql script file using heredoc on VM1, then ran it with: mysql -h 192.168.56.20 -u alms_user -p attendance_leave < ~/tables.sql

8. Conclusion

The ALMS Secure Web project successfully delivered a fully functional, security-hardened web-based HR management system. The deployment across two isolated VMs establishes a meaningful security boundary between the web tier and the database tier, ensuring the database is never directly reachable from any public network.

All four mandatory OWASP controls were implemented and tested. A01 Broken Access Control is enforced through session checks and role-based guards on every protected page. A02 Cryptographic Failures is mitigated through bcrypt password hashing and HTTPS-only access via TLS 1.2/1.3. A03 Injection is mitigated through session-controlled query parameters, password_verify() for login, and htmlspecialchars() output encoding for XSS prevention. A07 Authentication Failures are addressed through secure cookie flags, session regeneration on login, attempt throttling, and proper session destruction on logout.

The two-VM architecture ensures the database server is never directly reachable from any public network, significantly reducing the attack surface. The least-privilege database user (SELECT, INSERT, UPDATE, DELETE only – no DROP, GRANT, or ALTER) further limits the impact of any potential vulnerability.

Limitations and Future Work

- i. The self-signed SSL certificate generates browser warnings. Production deployment would use Let's Encrypt Certbot for a trusted certificate.
- ii. CSRF token protection should be added to all POST forms for more complete A01 coverage.
- iii. Two-factor authentication (2FA) for admin accounts would strengthen A07 compliance.
- iv. Automated vulnerability scanning with OWASP ZAP is recommended before any production deployment.

References

Apache Software Foundation. (2024). Apache HTTP server version 2.4 documentation. <https://httpd.apache.org/docs/2.4/>

Canonical Ltd. (2024). Ubuntu server guide. <https://ubuntu.com/server/docs>

Electronic Frontier Foundation. (2024). Certbot documentation. <https://certbot.eff.org/>

OWASP Foundation. (2021). OWASP Top Ten. <https://owasp.org/www-project-top-ten/>

PHP Group. (2024). password_hash – PHP manual. <https://www.php.net/manual/en/function.password-hash.php>

PHP Group. (2024). MySQLi – PHP manual. <https://www.php.net/manual/en/book.mysqli.php>

Oracle Corporation. (2024). MySQL 8.0 reference manual. <https://dev.mysql.com/doc/refman/8.0/en/>

Watt, A. (2014). Database Design (2nd ed.). Pressbooks. <https://opentextbc.ca/dbdesign01/>

Stallings, W. (2017). Cryptography and Network Security: Principles and Practice (7th ed.). Pearson.